Resume Analyzer and Job Matcher

CAI4002: Introduction to Artificial Intelligence | Group Project Progress Report

Team Members: Martin Dang, Tuan Khang Phan
July 9, 2026

1. Project Status Summary

The project is in progress and on track. Since the proposal, we have built a working end-to-end prototype of the Resume Analyzer and Job Matcher: the system loads a real resume, extracts a structured skill profile from it, parses a target job posting into required and preferred qualifications, computes a compatibility score, and returns a ranked list of concrete edits that would raise that score. We have also trained and evaluated our first two supervised models for resume category classification on a public dataset of 2,484 real resumes. This report presents the current state of the pipeline, quantitative results from our first experiments, a walkthrough of the working demo, and the division of work between the two team members.

All code, experiment scripts, figures, and results in this report are versioned in our GitHub repository (github.com/khangpt2k6/Intro_to_AI).

2. Dataset

We are using the public Resume Dataset from Kaggle (snehaanbhawal/resume-dataset), as planned in the proposal. It contains 2,484 real resumes collected from livecareer.com, provided as plain text and labeled into 24 job categories such as Information Technology, Finance, Engineering, and Healthcare. The dataset serves two roles in our system: it is the training corpus for the TF-IDF vector space and the category classifier, and it provides realistic test resumes for the matching demo. Using a public dataset keeps the project reproducible and avoids the privacy concerns of collecting real applicant data.

Total resumes	**2,484**
Job categories	24
Training split (80%)	1,987 resumes
Held-out test split (20%)	497 resumes
Largest / smallest category	120 (IT, Business Dev.) / 22 (BPO)

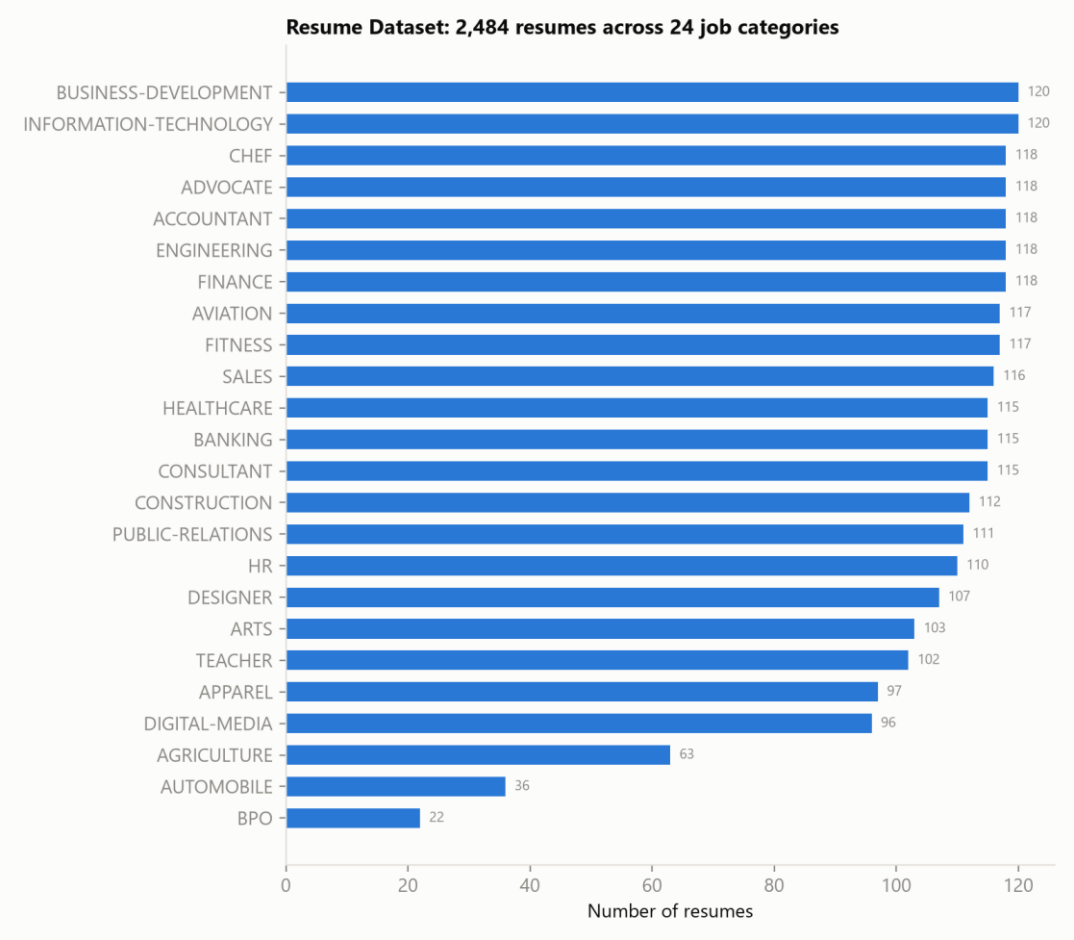

Figure 1. Distribution of the 2,484 resumes across the 24 job categories. The class imbalance (120 vs 22 resumes) is one challenge our classifier has to handle.

3. What We Have Built So Far

The pipeline from the proposal has five stages. Four of them are implemented and working; the table below shows the status of each.

Pipeline stage	Status	Implementation
Skill extraction	Working	A curated skill taxonomy (60+ canonical skills with aliases, e.g. 'JS' and 'JavaScript' normalize to one skill) matched with word-bounded regular expressions; each extracted skill carries a mention count and a confidence value.
Job description parsing	Working	Postings are split into hard constraints (required qualifications) and soft

		constraints (preferred qualifications), following the constraint-satisfaction framing in our proposal.
Compatibility scoring	Working	TF-IDF vectors (unigrams + bigrams, 20,000 features) fit on the full resume corpus; the score combines weighted skill coverage (60%) with the resume's percentile rank in corpus-wide cosine similarity to the posting (40%).
Recommendation engine	Working	Missing skills are ranked greedy best-first by the exact score gain each edit recovers; required skills carry twice the weight of preferred ones.
Resume category classification	First results	Naive Bayes and Logistic Regression over TF-IDF features, evaluated on a stratified held-out test set (Section 4).

Not yet implemented (planned before the final report): transformer embeddings (Sentence-BERT) as a second similarity signal alongside TF-IDF, PDF/DOCX text extraction for user-uploaded resumes, a simple web interface, and a Naive Bayes confidence model for individual extracted skills.

4. First Experimental Results

As a first supervised learning experiment, we trained two classifiers to predict a resume's job category from its text, using an 80/20 stratified train/test split (1,987 training, 497 test resumes). This classifier gives the analyzer a sense of which job family a resume currently signals, and it exercises the same TF-IDF representation the matcher uses. With 24 classes, random guessing would score about 4.2%.

Model	Accuracy	Macro F1
Multinomial Naive Bayes	57.3%	0.494
Logistic Regression	68.6%	0.637

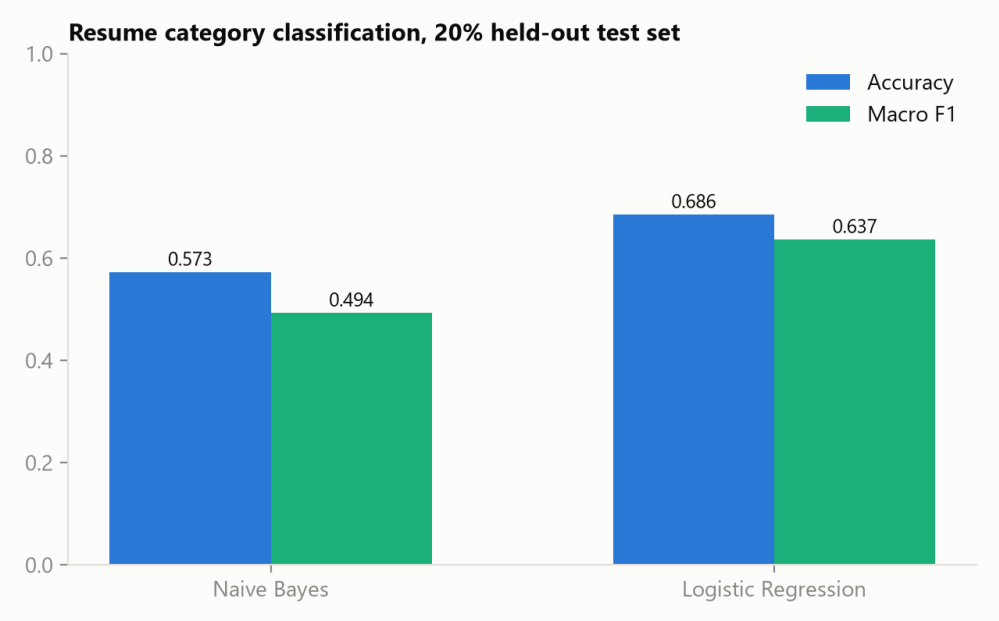

Figure 2. Both models beat the 4.2% random baseline by a wide margin; Logistic Regression is the stronger of the two.

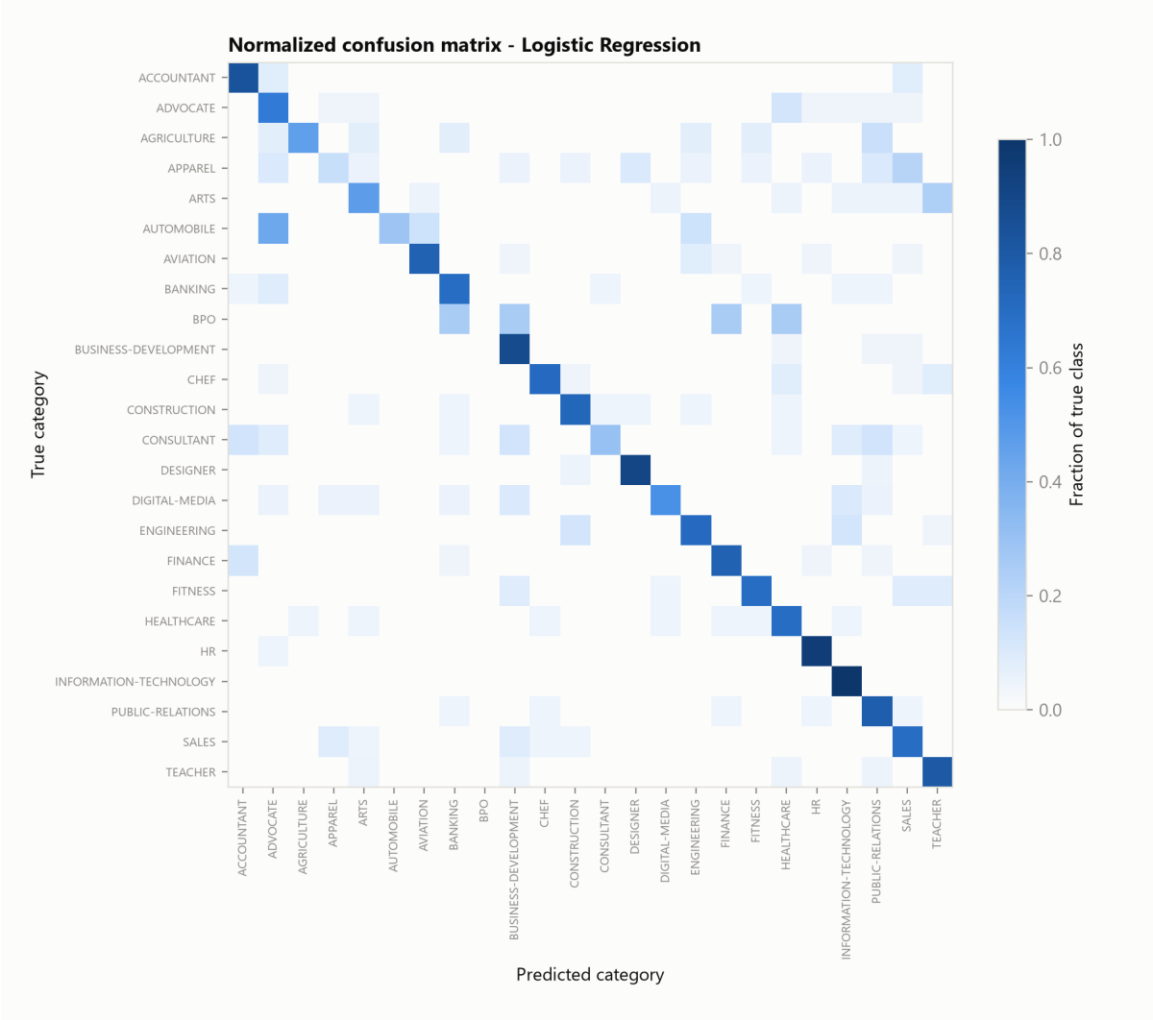

Figure 3. Normalized confusion matrix for Logistic Regression. The strong diagonal shows most categories are learned well; the remaining confusion concentrates in genuinely overlapping categories (e.g. Finance vs Accountant, Sales vs Business Development).

Takeaways: Logistic Regression outperforms Naive Bayes by about 11 points of accuracy, and the errors that remain are concentrated in semantically overlapping categories, which supports our plan to add dense transformer embeddings that capture context beyond keyword counts.

5. End-to-End Demo

To validate the whole pipeline, we matched one real Information Technology resume from the dataset (ID 83816738) against three job postings we wrote in realistic formats: a backend software engineer role, a data analyst role, and a staff accountant role. The system behaves the way a human reviewer would expect: the IT resume scores 82.8/100 against the software engineer posting, 49.0/100 against the data analyst posting, and only 4.0/100 against the accountant posting.

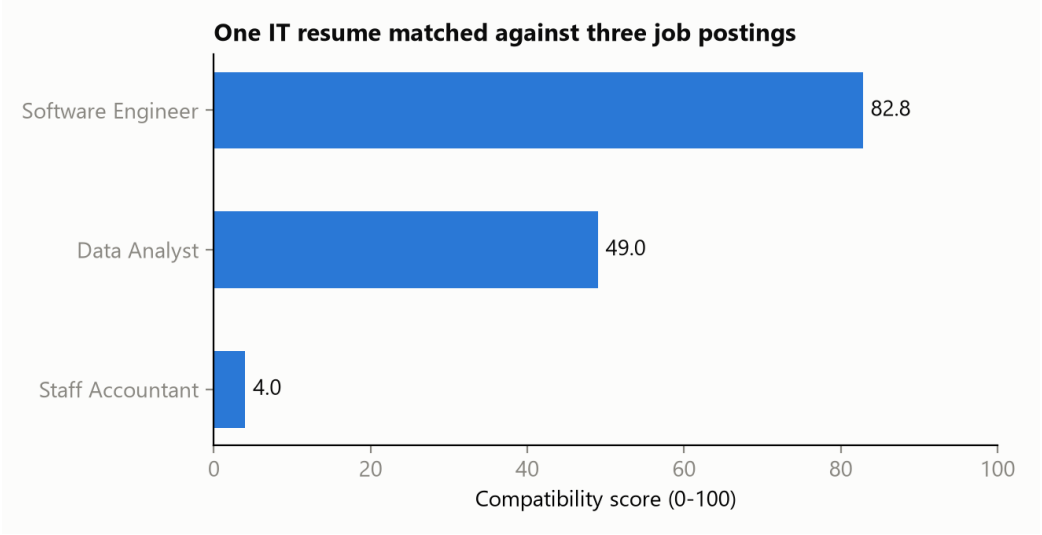

Figure 4. The same resume scored against three postings. The score separates the strong fit from the partial and poor fits.

For the software engineer posting, the analyzer matched 6 of 8 required skills (AWS, Agile, Git, Java, REST APIs, SQL) plus 3 preferred skills (CI/CD, Linux, React), and identified Docker, Python as the two missing required skills. The recommendation engine estimates that surfacing each one is worth about +5.7 points, so they are ranked first, ahead of preferred-skill edits like Kubernetes.

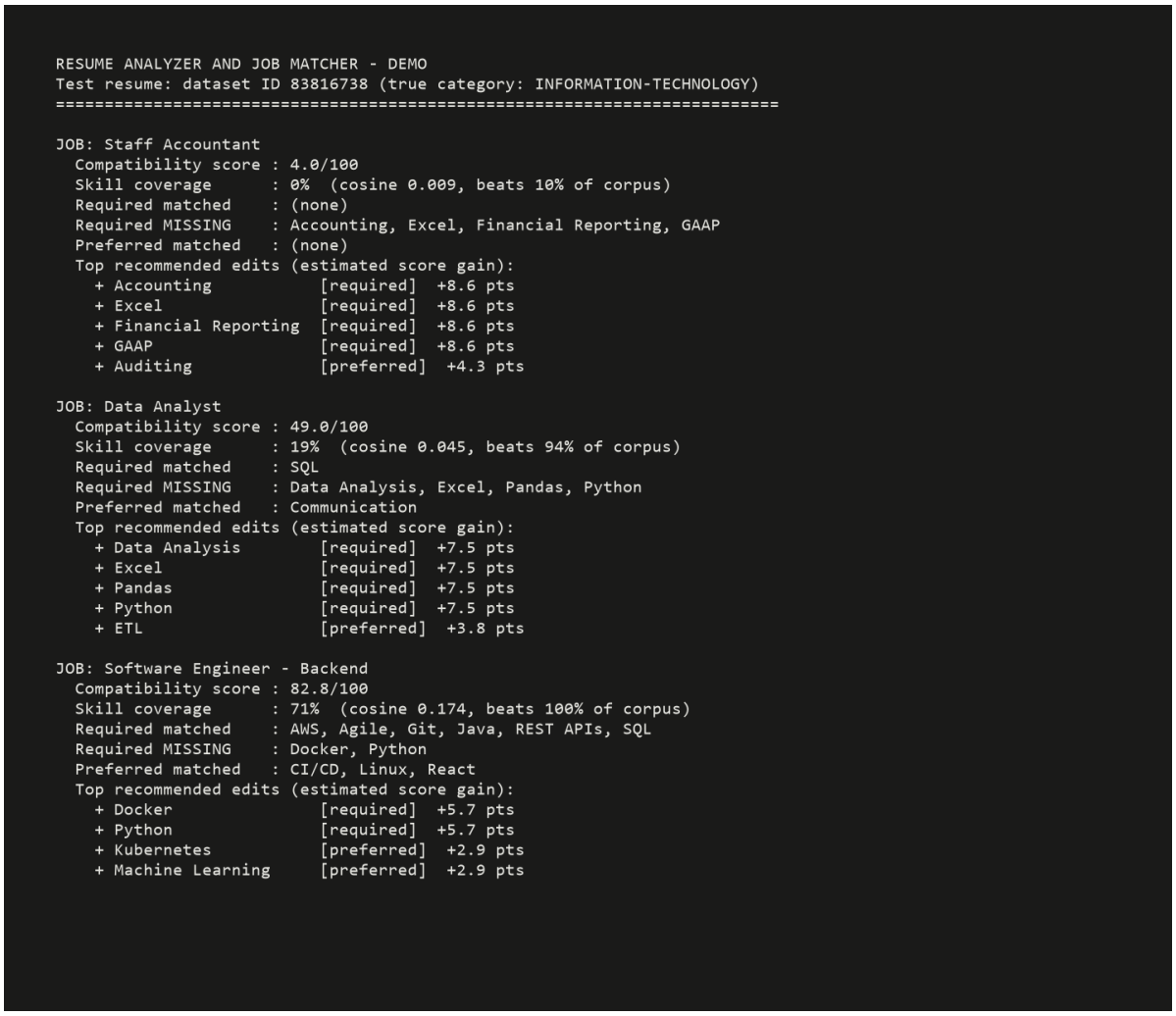

Figure 5. Screenshot of the analyzer's console output for the demo resume across all three postings, including the ranked edit recommendations.

6. Challenges Encountered

- **Class imbalance in the dataset.** The largest categories have 120 resumes while the smallest (BPO) has only 22, so a model can look accurate while ignoring small classes. We addressed this with a stratified train/test split and by reporting macro F1 (which weights every class equally) next to raw accuracy.
- **Semantically overlapping categories.** Much of the classification error concentrates in category pairs that genuinely share vocabulary, such as Finance vs Accountant and Sales vs Business Development. Naive Bayes suffered most from this, which is why we added Logistic Regression as a second learner (+11 points of accuracy) and plan to test transformer embeddings that capture context beyond shared keywords.
- **Raw cosine similarities are not interpretable.** Resume-to-posting cosine values in our corpus range from about 0.01 to 0.17, so presenting them directly would make every match look terrible. We solved this with percentile calibration: the score reports

where the resume ranks against all 2,484 corpus resumes for that same posting, which turns an opaque 0.174 into an interpretable 'beats 100% of the corpus'.

- **Short skill aliases cause false positives.** Aliases like 'R', 'Go', or 'C' match ordinary English words. We mitigated this with word-boundary regular expressions and by requiring longer context forms (for example 'R programming' instead of bare 'R'), at the cost of some recall; measuring that precision/recall trade-off on a hand-labeled sample is on our remaining-work list.
- **Noisy resume text.** The dataset resumes were converted from PDFs, so the text contains run-together section headers and inconsistent formatting. Our first demo subject extracted almost no skills for this reason; investigating it led us to broaden the alias lists and pick test subjects systematically instead of arbitrarily.

7. Team Member Contributions

We split the work evenly (50/50), pairing on design decisions and dividing implementation by pipeline stage:

Martin Dang (50%)	Tuan Khang Phan (50%)
Skill taxonomy design and alias normalization; skill extraction module; job description constraint parser (required vs preferred); recommendation engine and its ranking logic	Dataset acquisition and preprocessing; TF-IDF matching engine and percentile calibration; classifier experiments (Naive Bayes, Logistic Regression) and evaluation; demo script and figures
Joint: sample job postings, testing the pipeline end to end, this progress report	Joint: system design, error analysis of demo output, this progress report

8. Remaining Work

- Add Sentence-BERT embeddings as a dense similarity signal next to TF-IDF and compare the two on the same demo cases (connects to the deep learning material from class).
- Add PDF and DOCX text extraction so users can upload their own resume files (the dataset also ships the same resumes as PDFs, which gives us a ready-made test set for this).
- Replace the frequency-based skill confidence heuristic with a Naive Bayes estimate over the surrounding section text.
- Expand the skill taxonomy beyond the current 60+ canonical skills and evaluate extraction precision/recall on a hand-labeled sample.
- Build a minimal web interface that accepts a resume and a posting and renders the score, the matched/missing breakdown, and the ranked recommendations.

We are confident the remaining items fit in the time before the final deliverable, since the core pipeline and evaluation harness are already in place.